实验报告

系统开发工具基础（第四周）

姓名 甘如嫣
学号 25090032014
日期 2026年 9 月 12 日

目录

1 GitHub 仓库与提交记录

1.1 仓库链接

仓库地址：`https://github.com/ganruyan/system-basics-tools`

1.2 提交记录

本周材料按题目分组分次提交，未使用单次全量提交。

图 1: GitHub 提交记录

2 课上给助教检查的内容与结果

本节为课上完成、交由助教检查的四道课堂练习及其结果。

2.1 第 13 题：建立可执行的本地质量门禁

主题：代码质量 **建议用时：**12–15 分钟
复用 greetlab 包，在 `q13` 中建立一个轻量本地质量检查。

- 复制 `q10` 为 `q13`；在 `pyproject.toml` 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 `ruff format`、`ruff check` 和 `pytest`，修复全部问题，不得全局忽略规则。
- 编写 `check.sh`，依次执行 `ruff format --check`、`ruff check` 和 `pytest`；运行并保证返回 0。

结果：

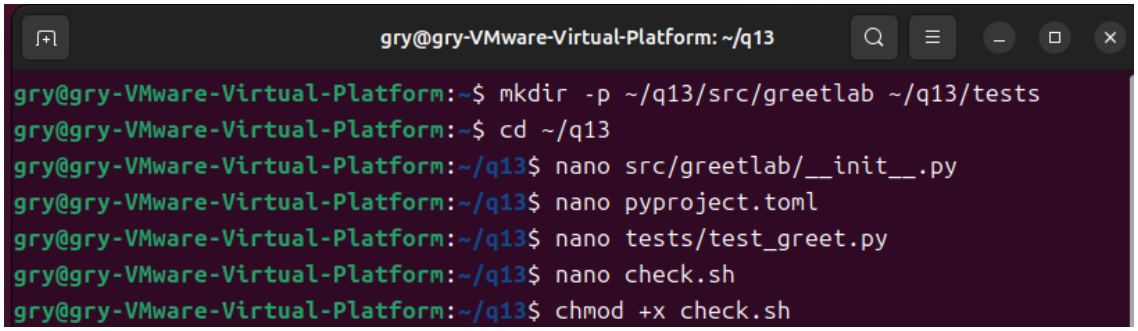

图 2: 第 13 题（a）ruff 与 pytest 检查结果

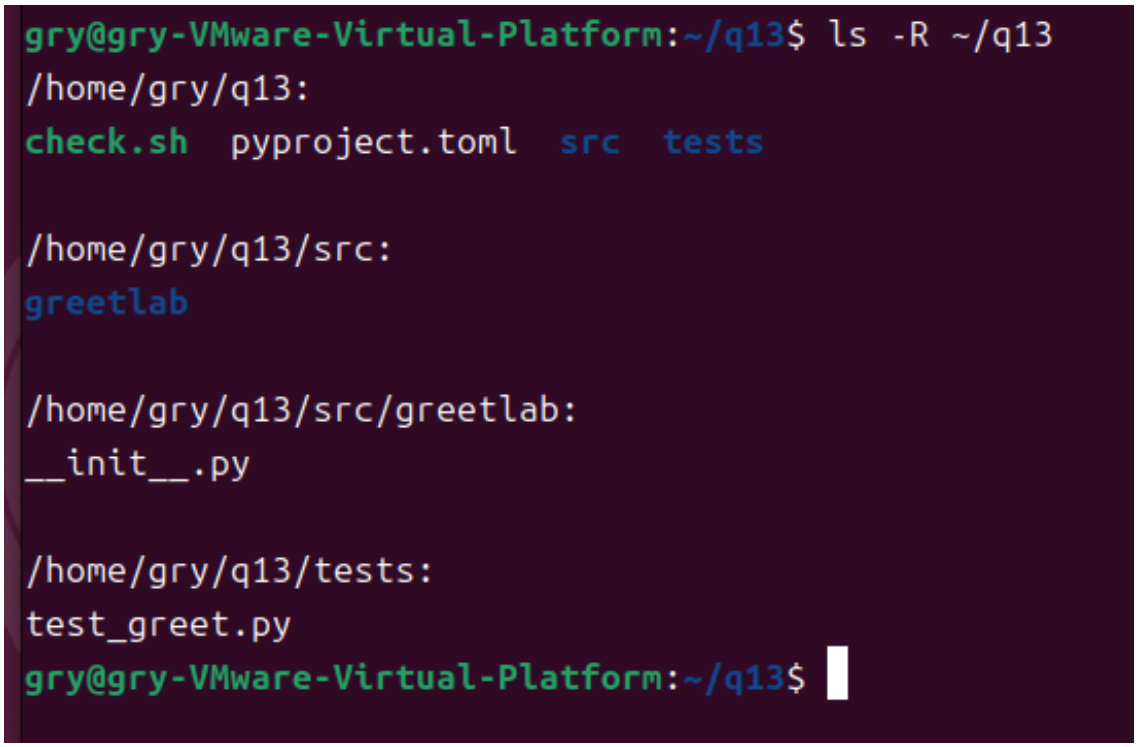

图 3: 第 13 题（b）check.sh 运行结果

2.2 第 14 题：让 Make 只重建真正受影响的产物

主题：元编程：构建系统 **建议用时：**12–15 分钟
在 `q14` 中按给定内容创建三个源文件，只需完成 Makefile。

- 创建给定的 `data.csv`、`stats.py`、`build_report.py` 和 `report.md`。
- 编写 Makefile，包含 `all`、`stats.txt`、`report.txt` 和 `clean`；准确列出数据文件、脚本和中间产物依赖，并把 `all`、`clean` 声明为 `.PHONY`。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- `touch data.csv` 后再次 make，确认 `stats.txt` 和 `report.txt` 重新生成；clean 只删除生成物。

结果：

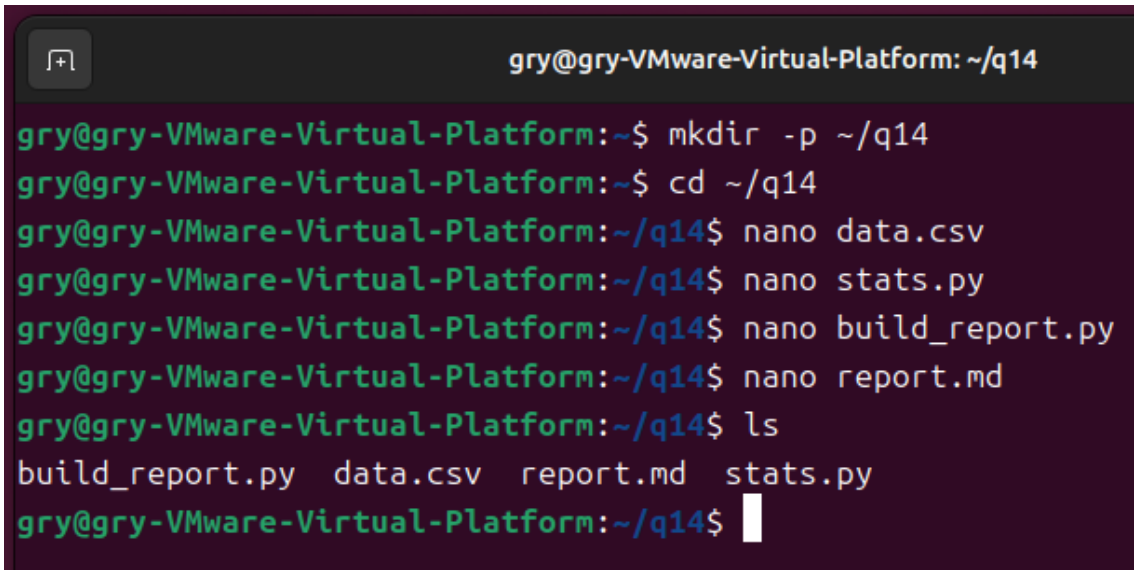

图 4: 第 14 题（a）Makefile 依赖与首次构建

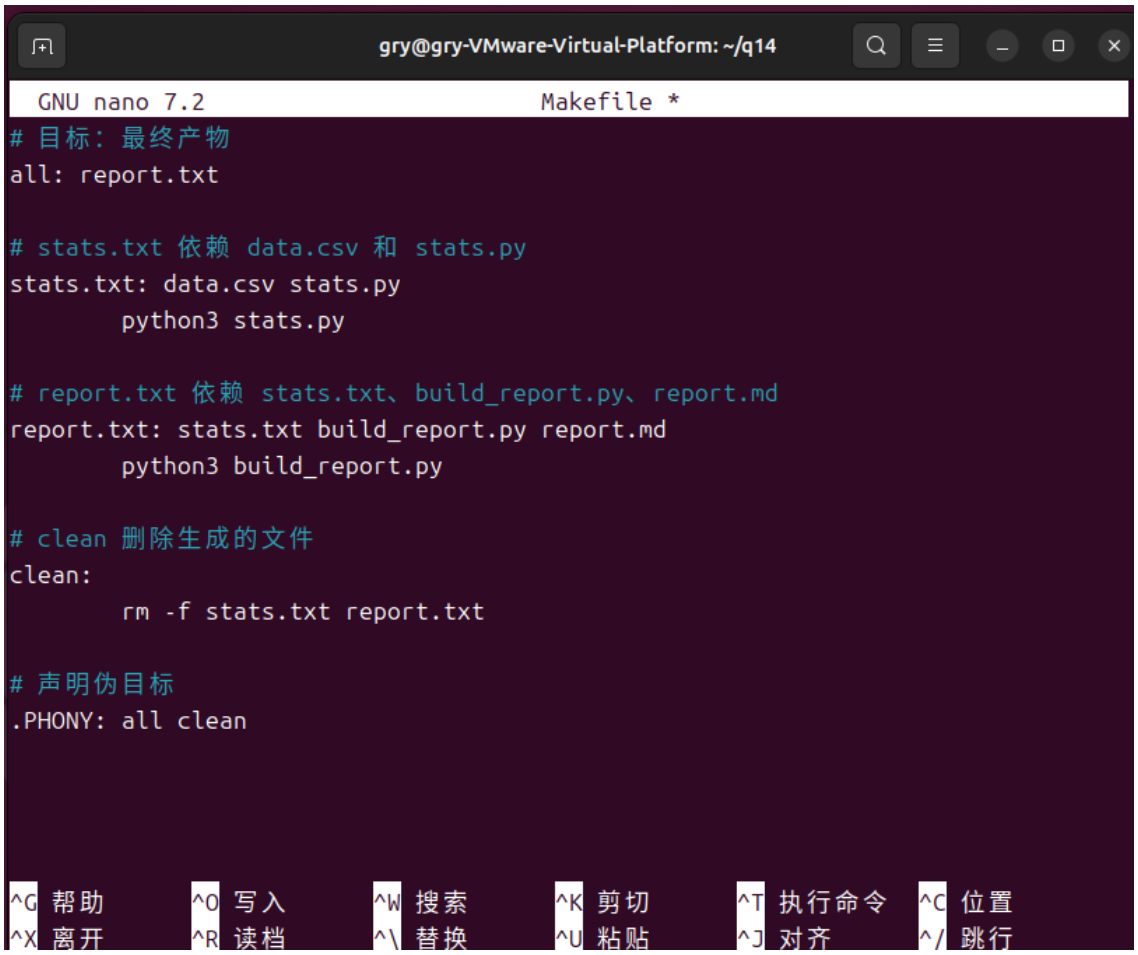

图 5: 第 14 题（b）增量构建与 clean 清理

2.3 第 15 题：把本地 API 数据转换为可读报告

主题： API、jq、CLI 约定与 Markdown **建议用时：** 12–15 分钟

在 q15 中创建给定 JSON 和本地 HTTP 服务，再用 curl 与 jq 生成简短 Markdown 报告。

- 把给定 JSON 保存为 `packages.json`，并在 q15 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。
- 使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录，并按 downloads 降序、name 升序排列。
- 编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 name/version/downloads 三列表格。

结果：

```
gry@gry-VMware-Virtual-Platform:~$ curl -fsSL https://mirrors.aliyun.com/docker-
ce/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-k
eyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docke
r-archive-keyring.gpg] https://mirrors.aliyun.com/docker-ce/linux/ubuntu $(lsb_r
elease -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
文件‘/usr/share/keyrings/docker-archive-keyring.gpg’已存在。 是否覆盖？(y/N)
```

图 6: 第 15 题（a）本地服务与 curl 获取数据

```
gry@gry-VMware-Virtual-Platform: ~/q15
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/q15
gry@gry-VMware-Virtual-Platform:~$ cd ~/q15
gry@gry-VMware-Virtual-Platform:~/q15$ nano packages.json
gry@gry-VMware-Virtual-Platform:~/q15$ ls
packages.json
gry@gry-VMware-Virtual-Platform:~/q15$ cat packages.json
[
{"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
{"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
{"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
{"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
{"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
gry@gry-VMware-Virtual-Platform:~/q15$
```

图 7: 第 15 题（b）jq 筛选与生成的 summary.md

2.4 第 16 题：修复并交付一个陌生的小型工具仓库

主题：课堂综合测试 **建议用时：**12–15 分钟
复用第 13 题的项目，在 `q16` 中完成一次 15 分钟综合检查。

- 复制 `q13` 为 `q16` 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 `Hello, name!`，运行 `check.sh` 确认测试失败；随后定位并修复。
- 编写最小 Makefile，包含 `check`、`build` 和 `clean`；check 调用 `check.sh`，build 生成 wheel。
- 运行 `make check` 和 `make build`，计算 wheel 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。

结果：

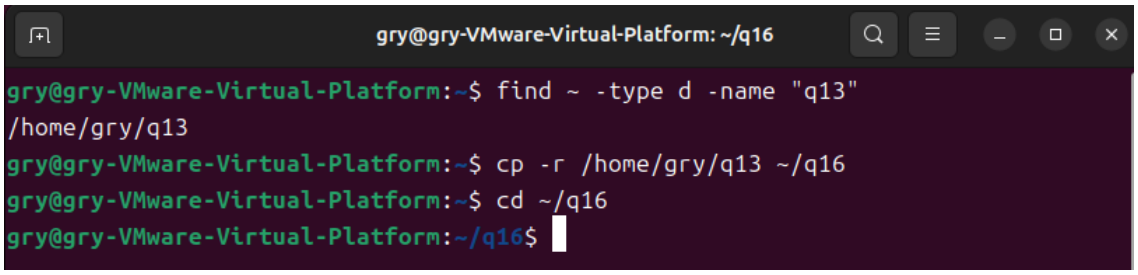

图 8: 第 16 题（a）make check 与 make build

```
gry@gry-VMware-Virtual-Platform:~/q16$ git init
提示：使用 'master' 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示：配置使用初始分支名，并消除这条警告，请执行：
提示：
提示：   git config --global init.defaultBranch <名称>
提示：
提示：除了 'master' 之外，通常选定的名字有 'main'、'trunk' 和 'development'。
提示：可以通过以下命令重命名刚创建的分支：
提示：
提示：   git branch -m <name>
已初始化空的 Git 仓库于 /home/gry/q16/.git/
gry@gry-VMware-Virtual-Platform:~/q16$ git status
位于分支 master

尚无提交

未跟踪的文件：
  （使用 "git add <文件>..." 以包含要提交的内容）
        check.sh
        pyproject.toml
        src/
        tests/

提交为空，但是存在尚未跟踪的文件（使用 "git add" 建立跟踪）
gry@gry-VMware-Virtual-Platform:~/q16$ nano src/greetlab/__init__.py
gry@gry-VMware-Virtual-Platform:~/q16$ ./check.sh
=====1. ruff format check =====
./check.sh: 行 4: ruff: 未找到命令
gry@gry-VMware-Virtual-Platform:~/q16$
```

图 9: 第 16 题（b）wheel 的 SHA-256 与最终提交

3 课后练习（11 个实例）

课后练习共完成 11 个实例，全部在 Ubuntu 虚拟机的终端中实际操作完成。

3.1 实例 1：正则与 grep 查找危险代码

题目：编写正则表达式，用 grep 在代码中查找 `subprocess.Popen(..., shell=True)` 的出现位置；随后故意“破坏”这个正则，观察 semgrep 是否仍能匹配到让 grep 失效的危险代码。

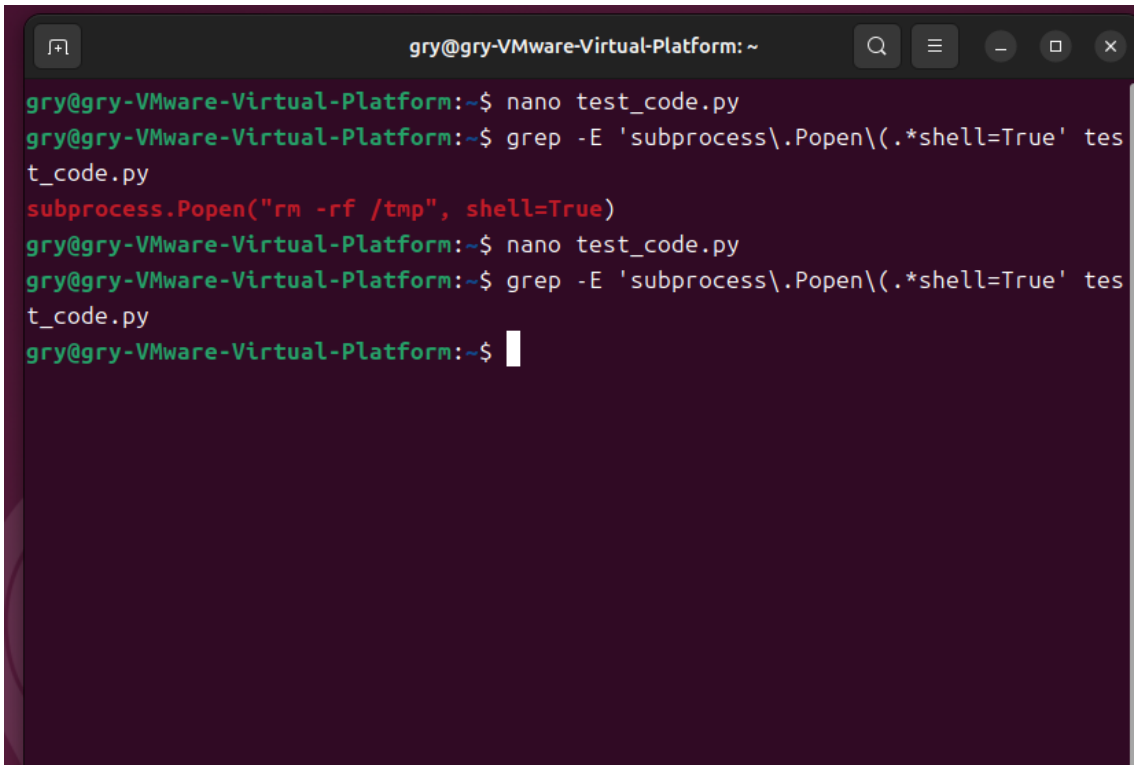

图 10: 实例 1：grep 与 semgrep 匹配结果

3.2 实例 2：正则提取 JSON 中的 name

题目：写一个 regex 从形如 `{"name": "Alyssa P. Hacker", "college": "MIT"}` 的 JSON 中捕获 name。第一次尝试容易把 `Alyssa P. Hacker", "college": "MIT` 一起提取出来，需要借助 Python regex 文档理解贪婪量词并修复。

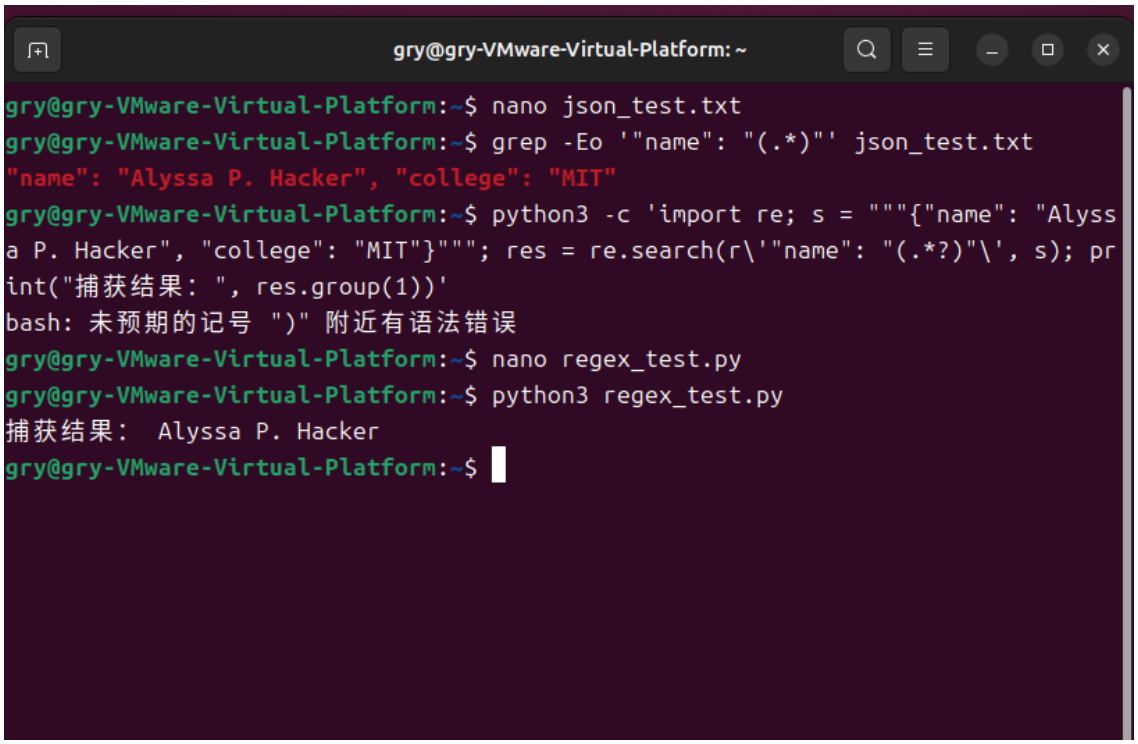

图 11: 实例 2：正则提取与修复后的结果

3.3 实例 3：black、flake8 与 pre-commit 钩子

题目：在本地 Git 仓库配置 black（格式化器）、flake8（linter）与 pre-commit 钩子。编写带格式错误、lint 错误的 Python 代码，验证提交时 black 自动修复格式；剩余 lint 错误交由编程辅助工具生成修复代码，人工对比 diff 确认没有破坏原有逻辑。

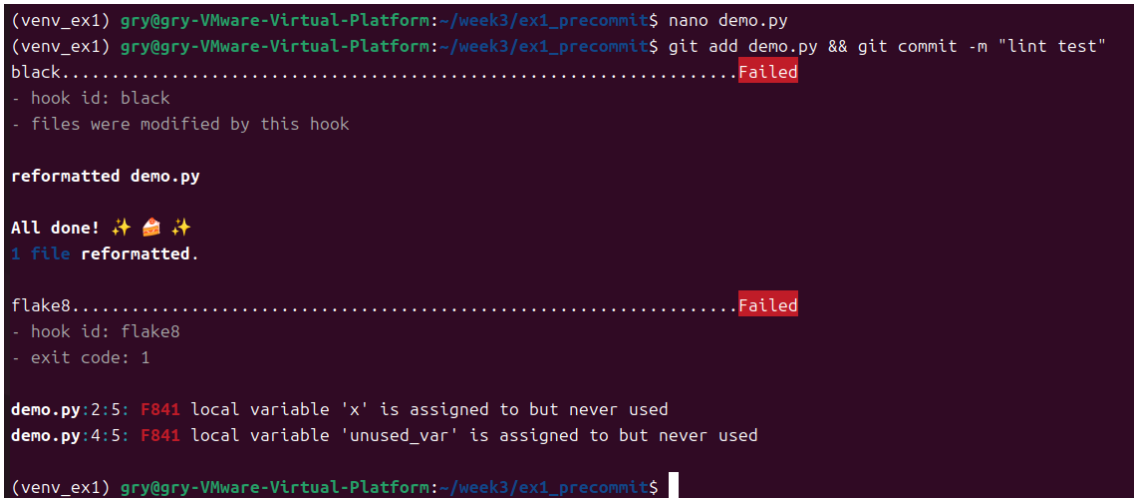

图 12: 实例 3（a）pre-commit 拦截与报错

```
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git diff demo.py
diff --git a/demo.py b/demo.py
index 7ef7ab4..38dd78d 100644
--- a/demo.py
+++ b/demo.py
@@ -1,5 +1,3 @@
-def hello(    name ):
-    x=  1
-    print(  "hi",name )
-    unused_var = 999 # flake8会报：变量定义了没使用
+def hello(name):
+    print("hi", name)

(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
```

图 13: 实例 3（b）修复后提交通过

3.4 实例 4：pytest 单元测试与覆盖率报告

题目：为项目编写单元测试，用 pytest-cov 生成 HTML 覆盖率报告并观察未覆盖的行；覆盖率较低时补充测试用例，观察覆盖率提升。

结果：初始只测试了加法和减法，覆盖率 62%，HTML 报告里 div 函数整段标红；补充除法的正常与除零异常两个用例后，4 个测试全部通过，覆盖率提升到 100%。

```
gry@gry-VMware-Virtual-Platform:~$ cd ~/week3/ex1_precommit
gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ source venv_ex1/bin/activate
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ pip install pytest pytest-cov
Requirement already satisfied: pytest in ./venv_ex1/lib/python3.12/site-packages (9.1.1)
Requirement already satisfied: pytest-cov in ./venv_ex1/lib/python3.12/site-packages (7.1.0)
Requirement already satisfied: iniconfig>=1.0.1 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=22 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (26.3)
Requirement already satisfied: pluggy<2,>=1.5 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in ./venv_ex1/lib/python3.12/site-packages (from pytest) (2.21.0)
Requirement already satisfied: coverage>=7.10.6 in ./venv_ex1/lib/python3.12/site-packages (from coverage[toml]>=7.10.6
->pytest-cov) (7.16.0)
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano test_calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ pytest --cov=calc --cov-report=html
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/gry/week3/ex1_precommit
plugins: cov-7.1.0
collected 2 items

test_calc.py ..                                                          [100%]

================================ tests coverage ================================
_____________ coverage: platform linux, python 3.12.3-final-0 _____________

Coverage HTML written to dir htmlcov
============================== 2 passed in 0.04s ===============================
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
```

图 14: 实例 4（a）初始覆盖率 62%

图 15: 实例 4（b）补充测试后覆盖率 100%

3.5 实例 5：持续集成流水线

题目：搭建持续集成流水线，在 push 阶段自动触发 “格式校验 → 静态检查 → 单元测试” 三级质量门禁，任一环节失败即阻断推送。人为注入格式错误与 Lint 违规，验证门禁能拦截问题代码；修复后验证流水线恢复正常。

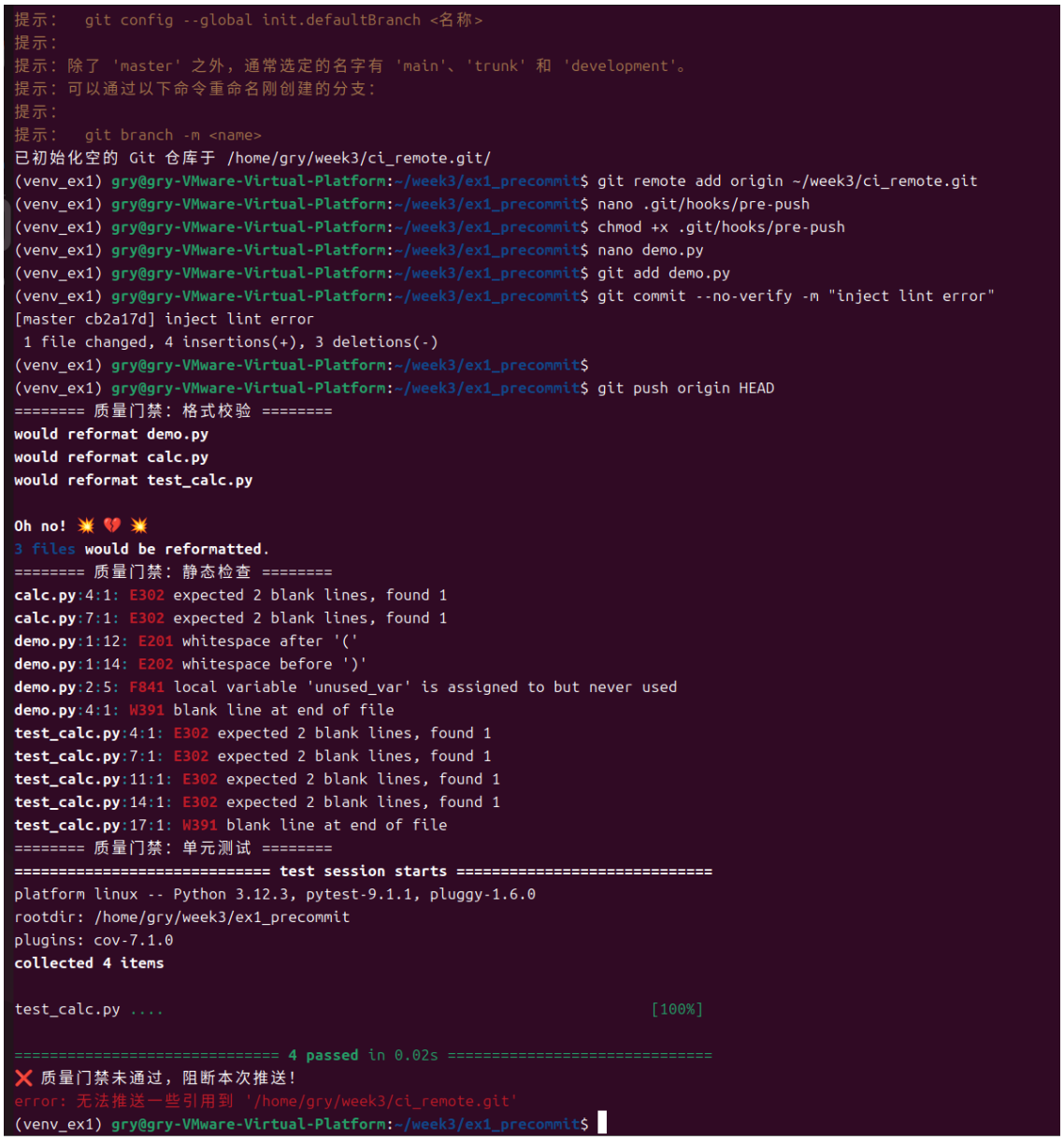

图 16: 实例 5（a）注入错误后门禁拦截推送

```
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ black calc.py test_calc.py
 demo.py
reformatted calc.py
reformatted test_calc.py

All done! ✨ 🍰 ✨
2 files reformatted, 1 file left unchanged.
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ nano test_calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ flake8 calc.py test_calc.p
y demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git add calc.py test_calc.
py demo.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git commit -m "fix all for
mat issues"
black....................................................................Passed
flake8...................................................................Passed
[master fa827ee] fix all format issues
 3 files changed, 33 insertions(+), 3 deletions(-)
 create mode 100644 calc.py
 create mode 100644 test_calc.py
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$ git push origin HEAD
======== 质量门禁：格式校验 ========
All done! ✨ 🍰 ✨
3 files would be left unchanged.
======== 质量门禁：静态检查 ========
======== 质量门禁：单元测试 ========
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/gry/week3/ex1_precommit
plugins: cov-7.1.0
collected 4 items

test_calc.py ....                                                        [100%]

================================== 4 passed in 0.01s ===================================
✅ 三级门禁全部通过，推送成功
枚举对象中：14，完成.
对象计数中：100% (14/14)，完成.
使用 4 个线程进行压缩
压缩对象中：100% (12/12)，完成.
写入对象中：100% (14/14)，1.52 KiB | 1.52 MiB/s，完成.
总共 14（差异 0），复用 0（差异 0），包复用 0
To /home/gry/week3/ci_remote.git
 * [new branch]      HEAD -> master
(venv_ex1) gry@gry-VMware-Virtual-Platform:~/week3/ex1_precommit$
```

图 17: 实例 5（b）修复后三级门禁通过

3.6 实例 6：正则查找替换 Markdown 项目符号

题目：练习 regex 查找替换，把讲义中的 - 项目符号替换为 *。直接替换所有 - 是不正确的，因为文件中 - 还有日期、连字符等非项目符号用途；用 ^- 限定行首的列表标记。

图 18: 实例 6（a）替换前 grep 查看多种用途

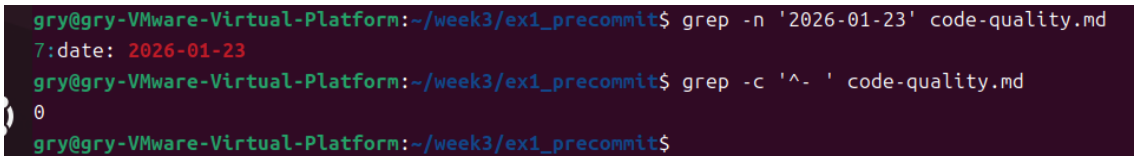

图 19: 实例 6（b）替换后验证普通横线未被改动

3.7 实例 7：Makefile 的 clean 构建目标

题目：在 paper.pdf 的 Makefile 中实现 clean 目标，用 `rm -f` 清理生成的 PDF 与辅助文件，并将构建目标声明为.PHONY；可用 `git ls-files` 区分源文件与生成物。

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git add Makefile paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git ls-files
Makefile
paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ make
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2023-11-01> patch level 1
L3 programming layer <2024-01-22>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2023/05/17 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdftex.def)
No file paper.aux.
[1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}] (./paper.aux) )</usr/sha
re/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr10.pfb>
Output written on paper.pdf (1 page, 14132 bytes).
Transcript written on paper.log.
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ ls
Makefile  paper.aux  paper.log  paper.pdf  paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$
```

图 20: 实例 7（a）make 构建生成产物

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ make clean
rm -f paper.pdf paper.aux paper.log
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ ls
Makefile  paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$
```

图 21: 实例 7（b）make clean 清理产物

3.8 实例 8：Cargo 依赖版本约束语法

题目：学习 Rust 构建系统的依赖管理，对每种语法（尖号、波浪号、通配符、比较、多个版本要求）构建一个实际场景。在 Cargo.toml 中为 serde、rand、toml、regex、log 分别配置 `^1.0.188`、`~0.8.5`、`0.8.*`、`>=1.7.0`、`>=0.4.14, <0.5` 并注释场景含义。

```
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/week3/cargo_lab
gry@gry-VMware-Virtual-Platform:~$ cd ~/week3/cargo_lab
gry@gry-VMware-Virtual-Platform:~/week3/cargo_lab$ nano Cargo.toml
gry@gry-VMware-Virtual-Platform:~/week3/cargo_lab$ cat Cargo.toml
[package]
name = "version_lab"
version = "0.1.0"
edition = "2021"

[dependencies]
# 1. 尖号 ^ ：^1.2.3 表示 >=1.2.3 且 <2.0.0（同一主版本内自动用新版本）
#    场景：serde 的 1.x API 稳定，希望自动获得补丁修复，又不怕主版本升级破坏代码
serde = "^1.0.188"

# 2. 波浪号 ~ ：~0.8.5 表示 >=0.8.5 且 <0.9.0（只允许补丁更新）
#    场景：rand 0.8.x 经过验证是稳定的，不希望 0.9 的大改动影响已有代码
rand = "~0.8.5"

# 3. 通配符 * ：0.8.* 表示 0.8 内的任意版本
#    场景：只关心主次版本，具体补丁号无所谓，有就用最新的
toml = "0.8.*"

# 4. 比较运算符：>=1.7.0 表示至少这个版本
#    场景：regex 从 1.7.0 开始才有需要的某个特性，版本不能更低
regex = ">=1.7.0"

# 5. 多个版本要求组合：逗号同时限制上下界
#    场景：版本不能太低（缺功能），也不能太高（可能有破坏性变更）
log = ">=0.4.14, <0.5"
gry@gry-VMware-Virtual-Platform:~/week3/cargo_lab$
```

图 22: 实例 8：Cargo.toml 五种版本约束及场景

3.9 实例 9：pre-commit 钩子构建检查

题目：编写 pre-commit 钩子，在提交前执行 `make paper.pdf`，构建失败则拒绝提交，避免产生包含不可构建版本的提交。

结果：故意把 `\begin{document}` 拼错为 `\begin{doc}`，提交时 pdflatex 报 LaTeX Error，钩子终止提交；修复后再次提交成功。

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git config user.name
gry
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git config user.email
179649847@qq.com
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ nano .git/hooks/pre-commit
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ chmod +x .git/hooks/pre-commit
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ nano paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git add paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git commit -m "test broken build"
==== 提交前构建检查 ====
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2023-11-01> patch level 1
L3 programming layer <2024-01-22>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2023/05/17 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))

! LaTeX Error: Environment doc undefined.

See the LaTeX manual or LaTeX Companion for explanation.
Type  H <return>  for immediate help.
 ...

l.2 \begin{doc}

?
! Emergency stop.
 ...

l.2 \begin{doc}

!  ==> Fatal error occurred, no output PDF file produced!
Transcript written on paper.log.
make: *** [Makefile:2: paper.pdf] 错误 1
❌ 构建失败，拒绝提交！
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$
```

图 23: 实例 9（a）构建失败拒绝提交

```
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ nano paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git add paper.tex
gry@gry-VMware-Virtual-Platform:~/week3/make_clean$ git commit -m "fix paper"
==== 提交前构建检查 ====
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.25 (TeX Live 2023/Debian) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2023-11-01> patch level 1
L3 programming layer <2024-01-22>
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2023/05/17 v1.4n Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/l3backend/l3backend-pdftex.def)
No file paper.aux.
[1{/var/lib/texmf/fonts/map/pdftex/updmap/pdftex.map}] (./paper.aux) )</usr/sha
re/texlive/texmf-dist/fonts/type1/public/amsfonts/cm/cmr10.pfb>
Output written on paper.pdf (1 page, 14132 bytes).
Transcript written on paper.log.
✅ 构建成功，允许提交
[master （根提交） a108636] fix paper
 2 files changed, 11 insertions(+)
 create mode 100644 Makefile
 create mode 100644 paper.tex
```

图 24: 实例 9（b）修复后提交成功

3.10 实例 10：shellcheck 检查 shell 脚本

题目：用 shellcheck 对仓库中所有 shell 文件做静态检查。在 test.sh 中故意写入等号两侧空格、变量未加双引号等错误，shellcheck 报出 SC1007、SC2086 等规则；修复后检查无输出。

```
gry@gry-VMware-Virtual-Platform:~$ mkdir -p ~/week3/shell_lab
gry@gry-VMware-Virtual-Platform:~$ cd ~/week3/shell_lab
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ nano test.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ shellcheck test.sh

In test.sh line 2:
name= "test"
     ^-- SC1007 (warning): Remove space after = if trying to assign a value (for empty string, use var='' ... ).


In test.sh line 3:
echo $name
     ^---^ SC2154 (warning): name is referenced but not assigned.
     ^---^ SC2086 (info): Double quote to prevent globbing and word splitting.

Did you mean:
echo "$name"

For more information:
  https://www.shellcheck.net/wiki/SC1007 -- Remove space after = if trying to...
  https://www.shellcheck.net/wiki/SC2154 -- name is referenced but not assign...
  https://www.shellcheck.net/wiki/SC2086 -- Double quote to prevent globbing ...
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$
```

图 25: 实例 10（a）shellcheck 报错

```
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ nano test.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ shellcheck test.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$ shellcheck *.sh
gry@gry-VMware-Virtual-Platform:~/week3/shell_lab$
```

图 26: 实例 10（b）修复后批量检查通过

3.11 实例 11：proselint 检查 Markdown 文档

题目：对仓库中所有.md 文件执行 proselint 写作风格检查。在 test.txt 中写入含 `very`、`utilize` 等不规范用词的句子，proselint 报出 weasel_words.very 和 misc.greylist 规则；干净文本检查后无告警。

```
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$ proselint check article.md
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$
```

图 27: 实例 11（a）proselint 报错

```
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$ nano test.txt
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$ proselint check test.txt
/home/gry/week3/proselint_lab/test.txt:1:7: weasel_words.very: Substitute 'damn' every time you're inclined to write 'very'; your editor will delete it and the writing will be just as it should be.
/home/gry/week3/proselint_lab/test.txt:1:106: misc.greylist: Use of 'utilize'. Do you know anyone who needs to utilize the word utilize?
(venv_pro) gry@gry-VMware-Virtual-Platform:~/week3/proselint_lab$
```

图 28: 实例 11（b）干净文本检查通过

4 解题感悟

这一周的内容明显比前面更接近真实工程：格式化、lint、测试覆盖率、持续集成，每一项都是在给代码上“保险”。我的第一个坑还是老问题——环境。装 pre-commit 时 Ubuntu 直接报 `externally-managed-environment`，全局 pip 装不了，后来才想到用虚拟环境解决；这类问题看报错就能定位，但第一次遇到时还是会懵。

做覆盖率练习时感受最明显：一开始只写了加法和减法的测试，跑出来只有 62%，HTML 报告里 div 函数整段标红，我才直观理解“覆盖率低意味着代码没被跑过”。补上除法的正常和除零两个用例后到 100%，但这个 100% 也提醒我，覆盖率只代表执行到了，不代表测对了边界，真正要补的其实是负数、精度这类情况。

还有几个容易忽略的小坑：proselint 新版本命令格式变成了 `proselint check`，直接按旧教程敲文件名会报错；Makefile 里命令前面必须是 Tab 而不是空格；git 钩子写好后要 `chmod +x` 才会执行。这些错误本身都不难，难的是报错信息一出来能不能沉住气一条条读。

这周最大的收获是把前面的零散命令串成了“质量门禁”的思路：格式化器管排版、linter 管规范、测试管功能，再靠 pre-commit 和 CI 在提交和推送时自动把关。做 AI 辅助修

复的练习时我也意识到，工具给的代码只能当草稿，每一步改动都要自己 diff 检查，确认没有破坏原有逻辑才能提交。